

How to Run This Project

Environment setup, data requirements, and the full CLI — verified end-to-end

Operator's guide · 2026-08-11

This is not a description of the pipeline — every command below was actually run against this checkout on 2026-08-11 (Python 3.12.3, venv already installed, GPU `cuda:0` visible) before being written down. Where numbers appear (row counts, timings, memory), they are the real output of that run, not estimates.

Contents. 1 What this is · 2 Prerequisites · 3 Data you need · 4 Environment setup · 5 Verifying the install · 6 The CLI (all 8 subcommands) · 7 A verified end-to-end run · 8 Generating the written reports · 9 Inspecting data · 10 Gotchas & troubleshooting · 11 Cheat sheet.

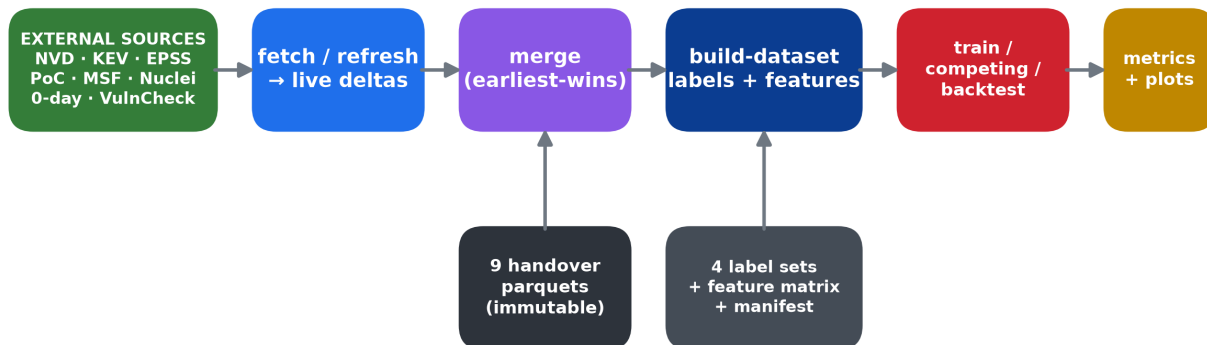
1. What This Is

A survival-analysis modeling layer that predicts **when** a published CVE becomes publicly weaponized (time-to-event), built on a pre-extracted multi-source CVE timeline dataset. It complements EPSS (which predicts in-the-wild exploitation probability over a fixed 30-day window) by characterizing the upstream weaponization pipeline: PoC → Metasploit/Nuclei → KEV/0-day.

Two layers, kept strictly separate (this matters for where files live and what you edit):

- `dataset_extraction-20260608T210903Z-3-002/dataset_extraction/` — immutable handover/source material (the extraction + enrichment scripts that produced nine parquet files under `out/`). Never modify this to change modeling behavior.
- `src/temporal_exploit/` — the modeling package. Reads the handover parquets, never writes to them. Generated outputs go to `artifacts/` (gitignored).

Data-flow pipeline — the spine is `cli.py` (7 subcommands)



Leakage firewall: build-dataset emits only publication-time-knowable features; every family is logged in `feature_provenance.csv`.

Figure 1. The data-flow pipeline. `cli.py` is the spine (eight subcommands, detailed in §6); `build-dataset` is the integration hub that emits four label sets, the feature matrix, and a content-hashed manifest. Source: `docs/architecture_flow.md`.

2. Prerequisites

Requirement	Verified value on this machine
OS	WSL2 Ubuntu (migrated off Windows/OneDrive 2026-06-12)
Python	3.12 (checked: 3.12.3, via the repo <code>.venv</code>)
Package manager	<code>uv</code> (faster than <code>pip</code> ; this is the standing preference for this repo)
Disk	~4 GB free minimum for <code>epss_history-001.parquet</code> alone; 753 GB free measured
RAM budget	≤ 6–8 GB hard ceiling — WSL2 caps well below the host's 16 GB
GPU (optional)	CUDA device for xgb/deep models — verified present: RTX 4060 Laptop, 8188 MiB VRAM
VRAM budget	≤ 7 GB — xgb (GPU AFT) and DeepSurv/DeepHit use this; CPU models (<code>cox</code> , <code>rsf</code>) don't need it

Memory is the binding constraint on this machine, not compute. Always run `free -g` before a heavy model fit. The 375M-row EPSS history file is the main risk (see §10) — it is read via streaming batches, never loaded whole.

3. Data You Need

All required data already lives in this repo checkout — there is nothing to download for a standard run. Two locations:

Location	Contents	Size
dataset_extraction-20260608T210903Z-3-002/ dataset_extraction/out/	8 handover parquets: cve_corpus, poc_dates, metasploit_dates, nuclei_dates, kev_events, google_0day, technique_cwe_chain, vrs_presence	tens of MB each
epss_history-001.parquet (repo root)	375M-row EPSS score history, one row group per day	3.7 GB (3,965,667,171 bytes)

These are treated as **immutable reproducibility material** — the extraction pipeline that produced them (from a VRS MongoDB dump) lives alongside them but is not something you re-run for normal modeling work. If you're starting from a fresh clone and these files are missing, that's the handover package to obtain separately; this guide assumes they're already present, as they are in this checkout.

4. Environment Setup

One-time, from the repo root:

```
uv venv --python 3.12 .venv
uv pip install -p .venv/bin/python -e ".[dev,xgb,boost]"
```

What each extra unlocks (from `pyproject.toml` — pick what you actually need):

Extra	Adds	When you need it
(core, always installed)	pandas, pyarrow, numpy, scikit-learn, scikit-survival, lifelines, matplotlib, seaborn, ijson	always — build-dataset, Cox, RSF
dev	pytest, pytest-cov, ruff, pre-commit	running the test suite / linting
xgb	xgboost ≥ 2.0	train --models xgb (GPU AFT — the abundant first-weaponization head)
boost	hazardous (SurvivalBoost, Inria)	train-competing --boost only
deep	torch, pycox, torchtuples	train --deep or train-competing --deep-hit (heavy; kept out of dev/CI)

Per this repo's own guidance (and prior measurement): default modeling to the GPU path (xgb) rather than CPU-heavy RSF/GBM, and skip the [deep] extra unless you specifically need DeepSurv/DeepHit — it pulls torch and isn't worth it below roughly 1,200 training events.

Console script. After install, the entry point is available directly:

```
temporal-exploit --help
# equivalent to: .venv/bin/python -m temporal_exploit.cli --help
```

5. Verifying the Install

Run the full test suite (the FutureWarning gate is baked into pyproject.toml, so plain pytest just works):

```
.venv/bin/python -m pytest -q
```

Verified on this checkout, 2026-08-11: 402 passed, 4 skipped in 114.29s. The 4 skips are the torch-gated deep-model tests (expected without the [deep] extra).

Single test, if you're iterating on one module:

```
.venv/bin/python -m pytest tests/test_labels.py::test_name -v
```

6. The CLI — All 8 Subcommands

Every subcommand below was invoked with `--help` against this checkout to confirm the flags; nothing here is from memory or the source alone.

Subcommand	One-line purpose
build-dataset	Build modeling labels and features from the handover parquets — the integration hub
train	Train and evaluate survival models (cox/rsf/xgb/cure) against a built dataset
train-competing	Competing-risks / multi-state training report (Aalen-Johansen, cause-specific Cox)
fetch	Run one live connector (kev, epss, nvd, nuclei, poc, metasploit, zeroday, exploithub, vulncheck_kev, greynoise) and save to a live dir
merge	Merge live-fetched deltas onto the handover parquets
backtest	Rolling-origin walk-forward backtest of time-to-weaponization
triage	Emit a per-CVE state-aware triage table (the deployable downstream tool)
refresh	Refresh all keyless sources in one shot, downloading only what changed

6.1 build-dataset

The command every other workflow depends on — turns the handover parquets into labels + features.

```
.venv/bin/python -m temporal_exploit.cli build-dataset \  
  --out-dir dataset_extraction-20260608T210903Z-3-002/dataset_extraction/out \  
  --artifact-dir artifacts \  
  --snapshot-date 2026-03-14 \  
  [--cutoff-date 2024-01-01] # locks a train/test split when given  
  [--technique-chain PATH] [--epss-path PATH]  
  [--landmarks 7,30] # writes landmark_features_{L}d.parquet/offset  
  [--description-text] # leakage-safe NLP feats (112 MB col; opt-in)
```

See §7 for a full run of this command with real output.

6.2 train

```
temporal-exploit train \  
  --artifact-dir artifacts --cutoff-date 2024-01-01 \  
  --report-dir artifacts/reports/my_run \  
  --label-set {first_weaponization,in_wild} \  
  --models cox,rsf,xgb,cure # comma list  
  [--rsf-sample N] [--landmark L] [--deep] [--cure-recalibrate]
```

Which models for which label-set (from this project's own bake-off results): for `in_wild`, penalized Cox wins — the ceiling there is data-limited (~250–400 events), not model-limited, and RSF/GBM/mixture-cure all measure $\leq$ Cox prospectively. For `first_weaponization` (45,947+ events, abundant), `xgb` (GPU XGBoost-AFT) is the headline ranker. `--deep` needs the `[deep]` extra and is disqualified below ~1,200 events.

6.3 train-competing

```
temporal-exploit train-competing \  
  --artifact-dir artifacts --cutoff-date 2024-01-01 \  
  --report-dir artifacts/reports/competing_run \  
  [--snapshot-date 2026-03-14] # needed for transitions; omit to skip  
  [--boost] [--deep-hit] # need [boost] / [deep] extras respectively
```

6.4 backtest

```
temporal-exploit backtest \  
  --out-dir dataset_extraction-20260608T210903Z-3-002/dataset_extraction/out \  
  --artifact-dir artifacts --report-dir artifacts/reports/backtest \  
  --snapshot-date 2026-03-14 --start 2021-01-01 \  
  [--model {cox,xgb,cure}] [--label-set {first_weaponization,in_wild}]  
  [--min-followup-days N] [--recalibrate]    # held-out isotonic recal per origin
```

Each rolling origin trains only on what was knowable at that point and scores the next period — the honest temporal-validation discipline this project insists on (never random K-fold).

6.5 triage

```
temporal-exploit triage \  
  --out-dir dataset_extraction-20260608T210903Z-3-002/dataset_extraction/out \  
  --artifact-dir artifacts --report-dir artifacts/reports/triage \  
  --snapshot-date 2026-03-14 [--min-pub DATE] [--horizon DAYS]
```

Emits the deployable per-CVE risk table — needs `publication_features.parquet` + `modeling_labels.parquet` from a prior `build-dataset` run.

6.6 fetch / merge / refresh — pulling fresh live data

These extend the handover parquets with newer data than the frozen snapshot; skip them for a first run.

```
# one connector at a time:  
temporal-exploit fetch --source kev --live-dir data/live  
temporal-exploit fetch --source epss --live-dir data/live --date 2026-08-01  
temporal-exploit fetch --source vulncheck_kev --live-dir data/live \  
  --api-key $VULNCHECK_API_TOKEN  
  
# merge the live deltas onto the handover parquets:  
temporal-exploit merge \  
  --handover-dir dataset_extraction-20260608T210903Z-3-002/dataset_extraction/out \  
  --live-dir data/live --out-dir data/merged  
  
# or refresh every keyless source in one shot:  
temporal-exploit refresh --live-dir data/live \  
  [--cache-dir .fetch_cache] [--repo-dir nuclei] [--with-metasploit]  
  [--epss-date 2026-08-01] [--vulncheck-token $VULNCHECK_API_TOKEN] [--with-nvdplus]
```

--source accepts: kev, epss, nvd, nuclei, poc, metasploit, zeroday, exploithub, vulncheck_kev, greynoise. VulnCheck and GreyNoise need an API token (--api-key or the matching env var); the rest are keyless.

7. A Verified End-to-End Run

This is the exact command run against this checkout on 2026-08-11, with its real output.

```
.venv/bin/python -m temporal_exploit.cli build-dataset \
  --out-dir dataset_extraction-20260608T210903Z-3-002/dataset_extraction/out \
  --artifact-dir artifacts \
  --snapshot-date 2026-03-14 --cutoff-date 2024-01-01
```

Metric	Value
Wall time	19.8 s
Peak RSS	~1.27 GB (well inside the 6–8 GB budget)
cve_corpus rows processed	338,015
Event source composition	poc 186,709 · nuclei 4,137 · metasploit 3,107 · kev 1,542 · google_0day 344
in_wild observed events	1,543
Warning raised (expected)	event source 'poc' is 97.2% of observed events — exactly the framing caveat in §10

Files written to --artifact-dir:

File	Contents
modeling_labels.parquet	First-weaponization labels (338,015 rows)
in_wild_labels.parquet	In-wild labels (KEV + 0-day, PoC excluded)
per_signal_labels.parquet	Per-source event labels
competing_risks_labels.parquet	Multi-state / competing-risks labels
publication_features.parquet	Leakage-safe publication-time feature matrix
train_cve_ids.txt / test_cve_ids.txt	Locked time-based split (only written when --cutoff-date is given)
feature_provenance.csv	The leakage audit trail — one row per feature family
manifest.json	Content-hashed (sha256) manifest of every artifact above

Once this has run, point `train` / `backtest` / `trriage` at the same `--artifact-dir` (see §6.2–6.5). A typical next step:

```
temporal-exploit train --artifact-dir artifacts --cutoff-date 2024-01-01 \
  --report-dir artifacts/reports/my_run --label-set in_wild --models cox
```

8. Generating the Written Reports

Four standalone reportlab scripts render PDFs from the repo's docs + artifacts/*.json (so numbers can't drift from source). All run the same way — `venv python`, no arguments. Paths below are relative to `scripts/` and `docs/` respectively:

Script (in scripts/)	Output (in docs/)	Audience
build_project_report_pdf_2026-06-23.py	project_report_2026-06-23.pdf	Full technical report + architecture + 40-source reference list

build_meeting_handout_ 2026-06-30.py	meeting_update_ 2026-06-30.pdf	One-page supervisor progress handout
build_meeting_handout_fig_ 2026-06-30.py	meeting_update_2026-06-30_ with_figure.pdf	Same handout, with the EPSS-parity chart embedded
build_plain_explainer_ 2026-06-30.py	explainer_plain_language_ 2026-06-30.pdf	No-jargon explainer for a non-security audience
build_howto_run_pdf_ 2026-08-11.py	howto_run_ 2026-08-11.pdf	This document

```
.venv/bin/python scripts/build_project_report_pdf_2026-06-23.py
.venv/bin/python scripts/build_meeting_handout_2026-06-30.py
.venv/bin/python scripts/build_meeting_handout_fig_2026-06-30.py
.venv/bin/python scripts/build_plain_explainer_2026-06-30.py
.venv/bin/python scripts/build_howto_run_pdf_2026-08-11.py
```

Some pull figures from `docs/figures/*.png` (regenerate those first with `scripts/build_report_figures.py` if you've changed underlying artifacts) or from live artifacts `/*.json` — run a `training/backtest` command first if a figure or number looks stale or missing.

9. Inspecting Data Without Loading It

`view_parquet.py` reads schema and row counts from parquet metadata (no data load) and uses predicate pushdown for per-CVE filters — safe on the 375M-row EPSS history file.

```
cd dataset_extraction-20260608T210903Z-3-002/dataset_extraction
PY=../../.venv/bin/python
$PY view_parquet.py                # list available parquets
$PY view_parquet.py cve_corpus      # schema + stats + head(5)
$PY view_parquet.py kev_events --head 20
$PY view_parquet.py epss_history --cve CVE-2021-44228
$PY view_parquet.py cve_corpus --columns cve_id,published,description
$PY view_parquet.py epss_history --schema-only      # never loads the 3.7 GB file
$PY view_parquet.py cve_corpus --stats              # per-column null counts
```


10. Gotchas & Troubleshooting

- **Never add an `isin(cve_ids)` pushdown on `epss_history`.** It retains ~5–6 GB and breaches the memory budget. The only safe pushdown is by *date* (row-group-per-day), already wired in `_iter_epss_batches`.
- **Timezone mismatches raise, by design.** All handover date columns are `timestamp[ns, tz=UTC]`. Mixing tz-aware and naive timestamps raises on subtraction rather than silently producing wrong durations — if you hit this, add `utc=True` to your date parsing, don't strip the tz.
- **Verify column names against the actual parquet before assuming.** `cvss_v3_base_score` vs the real `cvss_v3_base` once silently produced all-zero features. Run `view_parquet.py <name> --schema-only` first.
- **List columns load as `numpy.ndarray`, not Python list.** Code touching them (`list_len`, `has_list_value`) must handle `ndarray`, not just `list`.
- **The 97.2% poc-dominance warning on build-dataset is expected, not a bug.** ~97% of first-weaponization events are public-PoC dates; only KEV and Google 0-day are true in-the-wild signals. A model trained on first-weaponization labels predicts time-to-public-tooling, not in-the-wild exploitation — don't reframe it as an EPSS competitor.
- **Check `free -g` before any heavy model fit**, especially RSF (slow, high RAM) — prefer `xgb` on GPU where the label-set supports it.
- **Mixture-cure (`--models cure`) is a documented dead-end** for the in-wild target: it looked promising on one split but a rolling-origin backtest overturned it (`IPA@180` → `-0.27`). The ~99.5%-censored target is administratively censored, not a cured population — don't revive it without seeing a genuine Kaplan-Meier plateau first.

11. Cheat Sheet

Task	Command
Set up env	<code>uv venv --python 3.12 .venv &&</code> <code>uv pip install -p .venv/bin/python -e ".[dev,xgb,boost]"</code>
Run tests	<code>.venv/bin/python -m pytest -q</code>
Build dataset	<code>temporal-exploit build-dataset --out-dir <handover/out> \</code> <code>--artifact-dir artifacts --snapshot-date <date> --cutoff-date <date></code>
Train in-wild	<code>temporal-exploit train --artifact-dir artifacts --cutoff-date <date> \</code> <code>--report-dir <dir> --label-set in_wild --models cox</code>
Train first-weap	<code>temporal-exploit train --artifact-dir artifacts --cutoff-date <date> \</code> <code>--report-dir <dir> --label-set first_weaponization --models xgb</code>
Backtest	<code>temporal-exploit backtest --out-dir <handover/out> --artifact-dir artifacts \</code> <code>--report-dir <dir> --snapshot-date <date> --start <date></code>
Triage table	<code>temporal-exploit triage --out-dir <handover/out> --artifact-dir artifacts \</code> <code>--report-dir <dir> --snapshot-date <date></code>
Inspect a parquet	<code>python view_parquet.py <name> --schema-only</code>
Check memory before a fit	<code>free -g</code>

Regenerate this PDF	<code>.venv/bin/python scripts/build_howto_run_pdf_2026-08-11.py</code>
---------------------	---

Full option reference: `temporal-exploit <subcommand> --help` for any of the 8 subcommands always reflects the current code — this document is a snapshot as of 2026-08-11; re-run `--help` if flags look off after a code change.